HTML, CSS & jQuery

A Beginner's Quick Reference Guide

Contents

I HTML Basics **3**

1 What is HTML? **3**

2 HTML Page Structure **3**

3 Common HTML Tags **3**

3.1 Headings . . . 3
3.2 Paragraphs and Text Formatting . . . 4
3.3 Links and Images . . . 4
3.4 Lists . . . 4
3.5 Div and Span . . . 4
3.6 Tables . . . 4
3.7 Forms . . . 5
3.8 Semantic Tags (HTML5) . . . 6

4 HTML Attributes **6**

II CSS Basics **6**

5 What is CSS? **6**

6 Three Ways to Add CSS **6**

7 CSS Syntax **7**

8 CSS Selectors (with Examples) **7**

8.1 Universal Selector . . . 7
8.2 Element (Type) Selector . . . 7
8.3 Class Selector . . . 8
8.4 ID Selector . . . 8
8.5 Grouping Selector . . . 8
8.6 Descendant Selector . . . 8
8.7 Child Selector . . . 9
8.8 Adjacent Sibling Selector . . . 9

8.9 General Sibling Selector . . . 9
8.10 Attribute Selectors . . . 9
8.11 Pseudo-class Selectors . . . 10
8.12 Pseudo-element Selectors . . . 10

9 CSS Properties Quick Reference 10
9.1 Colors and Backgrounds . . . 10
9.2 Text and Fonts . . . 10
9.3 Box Model . . . 11
9.4 Display and Visibility . . . 11
9.5 Positioning . . . 11
9.6 Flexbox (Simple Layout) . . . 11
9.7 CSS Variables . . . 12

III jQuery 12

10 What is jQuery? 12

11 Basic Syntax 12

12 Connecting jQuery to HTML and CSS 12

13 jQuery Selectors 13

14 Changing Content and Styles 13
14.1 Text and HTML . . . 13
14.2 CSS Styles . . . 13
14.3 Adding and Removing CSS Classes . . . 13

15 Adding and Removing Elements 14

16 Event Handling 14

17 DOM Traversal 14

18 Attribute and Property Manipulation 15

19 Form Handling 15

20 Table Handling 15

21 Chaining 16

22 jQuery vs. Vanilla JavaScript 16

23 Common Methods Cheat Sheet 16

24 Summary 17

Part I

HTML Basics

1 What is HTML?

HTML (HyperText Markup Language) is the skeleton of every web page. It describes the *structure* and *content* of a page using **tags**. A browser reads HTML and displays it visually.

Key facts

- Tags are written inside angle brackets: `<tagname>`
- Most tags come in pairs: an opening tag and a closing tag (`</tagname>`)
- Attributes add extra information inside the opening tag
- HTML alone has no color or style – that is CSS's job

2 HTML Page Structure

Every HTML file follows this skeleton:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Page</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <!-- Visible content goes here -->
    <h1>Hello World</h1>
    <p>This is a paragraph.</p>
</body>
</html>
```

- `<!DOCTYPE html>` – tells the browser this is HTML5
- `<head>` – metadata (not visible on the page)
- `<body>` – everything the user actually sees
- `<link>` – connects an external CSS file

3 Common HTML Tags

3.1 Headings

```
<h1>Largest Heading</h1>
<h2>Second Level</h2>
<h3>Third Level</h3>
<!-- h4, h5, h6 also exist -->
```

3.2 Paragraphs and Text Formatting

```
<p>This is a paragraph.</p>
<b>Bold text</b>
<i>Italic text</i>
<u>Underlined text</u>
<br>      <!-- line break (no closing tag) -->
<hr>      <!-- horizontal rule / divider line -->
<small>Smaller text</small>
<strong>Important text (also bold)</strong>
<em>Emphasized text (also italic)</em>
```

3.3 Links and Images

```
<!-- Link -->
<a href="https://example.com">Visit Example</a>
<a href="about.html" target="_blank">Opens in new tab</a>

<!-- Image -->
<img src="logo.png" alt="Site Logo" width="200">
```

3.4 Lists

```
<!-- Unordered (bullet) list -->
<ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>jQuery</li>
</ul>

<!-- Ordered (numbered) list -->
<ol>
    <li>Step One</li>
    <li>Step Two</li>
</ol>
```

3.5 Div and Span

```
<!-- div: block-level container (takes a full line) -->
<div id="box" class="card">
    <p>Content inside a div</p>
</div>

<!-- span: inline container (sits inside a line of text) -->
<p>My name is <span class="highlight">Ali</span>.</p>
```

3.6 Tables

```
<table border="1">
    <thead>
        <tr>
```

```
            <th>Name</th>
            <th>Age</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Ali</td>
            <td>25</td>
        </tr>
        <tr>
            <td>Sara</td>
            <td>22</td>
        </tr>
    </tbody>
</table>
```

3.7 Forms

```
<form id="myForm">
    <!-- Text input -->
    <label for="name">Name:</label>
    <input type="text" id="name" placeholder="Enter name">

    <!-- Password input -->
    <input type="password" id="pass" placeholder="Password">

    <!-- Email -->
    <input type="email" id="email" placeholder="you@email.com">

    <!-- Checkbox -->
    <input type="checkbox" id="agree"> I agree

    <!-- Radio buttons -->
    <input type="radio" name="plan" value="basic"> Basic
    <input type="radio" name="plan" value="pro"> Pro

    <!-- Dropdown -->
    <select id="country">
        <option value="pk">Pakistan</option>
        <option value="us">USA</option>
    </select>

    <!-- Textarea -->
    <textarea id="msg" rows="4" cols="30">Write here...</textarea>

    <!-- Buttons -->
    <button type="submit">Submit</button>
    <button type="reset">Reset</button>
</form>
```

3.8 Semantic Tags (HTML5)

Semantic tags describe the *purpose* of the content inside them, making pages easier to read and maintain.

```
<header>   Site logo and navigation bar   </header>
<nav>      Navigation links               </nav>
<main>     Main page content              </main>
<section>  A thematic section             </section>
<article>  A blog post or news item       </article>
<aside>    Sidebar content                </aside>
<footer>   Copyright and links            </footer>
```

4 HTML Attributes

Attributes provide extra information about an element and are always written inside the opening tag:

```
<p id="intro" class="lead" style="color:red;">Hello</p>
<a href="page.html" target="_blank" title="Go there">Link</a>
<img src="img.jpg" alt="Description" width="300" height="200">
<input type="text" placeholder="Type..." disabled>
```

Common attributes:

- `id` – unique identifier, used by CSS and jQuery
- `class` – group name, reused across many elements
- `style` – inline CSS (avoid for large projects)
- `href` – URL for links
- `src` – source file for images/scripts
- `alt` – alternative text for images
- `placeholder` – hint text inside form fields
- `disabled` – disables a form element

Part II

CSS Basics

5 What is CSS?

CSS (Cascading Style Sheets) controls the *look* of a web page – colors, fonts, spacing, borders, layout, and more. It is applied to the HTML elements selected by **CSS selectors**.

6 Three Ways to Add CSS

```
<!-- 1. External (recommended) -- link a separate .css file -->
<link rel="stylesheet" href="style.css">

<!-- 2. Internal -- inside a <style> tag in <head> -->
```

```
<style>
    p { color: blue; }
</style>

<!-- 3. Inline -- directly on the element (avoid for maintainability) -->
<p style="color: red;">Text</p>
```

7 CSS Syntax

```
selector {
    property: value;
    property: value;
}
```

Example:

```
p {
    color: navy;
    font-size: 16px;
    line-height: 1.6;
}
```

8 CSS Selectors (with Examples)

This is one of the most important topics in CSS. A selector tells the browser *which* element(s) to style.

8.1 Universal Selector

Matches *every* element on the page.

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

8.2 Element (Type) Selector

Matches all elements of a given tag name.

```
h1 {
    font-size: 32px;
    color: darkblue;
}

p {
    font-size: 16px;
    line-height: 1.5;
}
```

8.3 Class Selector

Matches any element that has the given class. Prefix with a dot (`.`).

```
<p class="intro">Welcome!</p>
<p class="intro">Another intro paragraph.</p>
```

```
.intro {
    color: teal;
    font-weight: bold;
}
```

One element can carry multiple classes: `class="card highlight"`

8.4 ID Selector

Matches exactly *one* element with the given id. Prefix with a hash (`#`).

```
<div id="header">Site Header</div>
```

```
#header {
    background-color: #333;
    color: white;
    padding: 20px;
}
```

IDs must be unique per page. Use classes for repeated patterns.

8.5 Grouping Selector

Apply the same rules to multiple selectors at once, separated by commas.

```
h1, h2, h3 {
    font-family: Arial, sans-serif;
    color: #222;
}

.btn, input[type="submit"] {
    background-color: dodgerblue;
    color: white;
}
```

8.6 Descendant Selector

Matches elements *inside* another element (any depth).

```
<div class="card">
    <p>This paragraph is inside .card</p>
</div>
<p>This paragraph is NOT inside .card</p>
```

```
.card p {
    color: purple;   /* only the <p> inside .card is affected */
}
```

8.7 Child Selector

Matches elements that are *direct* children only (one level deep). Uses >.

```
<ul id="menu">
    <li>Home</li>
    <li>
        <ul>
            <li>Sub-item</li>  <!-- NOT matched -->
        </ul>
    </li>
</ul>
```

```
#menu > li {
    list-style: none;
    font-weight: bold;
}
```

8.8 Adjacent Sibling Selector

Matches an element immediately after another. Uses +.

```
<h2>Title</h2>
<p>This paragraph comes right after the h2.</p>
<p>This one does NOT match.</p>
```

```
h2 + p {
    margin-top: 0;
    color: gray;
}
```

8.9 General Sibling Selector

Matches all siblings after an element (not just the immediate one). Uses ~.

```
h2 ~ p {
    color: gray;   /* all <p> elements after any <h2> */
}
```

8.10 Attribute Selectors

Match elements based on their attributes or attribute values.

```
input[type="text"]    { border: 1px solid blue; }  /* exact match   */
a[target="_blank"]    { color: orange; }            /* exact match   */
a[href^="https"]      { color: green; }             /* starts with   */
a[href$=".pdf"]       { color: red; }               /* ends with     */
a[href*="example"]    { font-weight: bold; }        /* contains      */
```

8.11 Pseudo-class Selectors

Style elements based on their *state* or *position*.

```
a:hover   { color: red; }          /* mouse over a link          */
a:visited { color: purple; }       /* already-visited link       */
a:active  { color: orange; }       /* link being clicked         */

input:focus   { outline: 2px solid blue; }  /* focused field     */
input:disabled{ opacity: 0.5; }             /* disabled field    */

li:first-child { font-weight: bold; }       /* first list item   */
li:last-child  { border-bottom: none; }     /* last list item    */
li:nth-child(2){ color: red; }              /* second list item  */
li:nth-child(odd) { background: #f0f0f0; } /* odd-indexed rows  */

p:not(.special) { color: gray; }            /* all <p> except .special */
```

8.12 Pseudo-element Selectors

Style a *part* of an element rather than the whole element.

```
p::first-line   { font-weight: bold; }       /* first line of text   */
p::first-letter { font-size: 2em; }          /* drop-cap effect       */

h2::before {
    content: ">> ";          /* inserts text before every <h2> */
    color: teal;
}

h2::after {
    content: " <<";          /* inserts text after every <h2>  */
    color: teal;
}
```

9 CSS Properties Quick Reference

9.1 Colors and Backgrounds

```
p {
    color: red;                       /* named color     */
    color: #e74c3c;                   /* hex code        */
    color: rgb(231, 76, 60);          /* RGB             */
    background-color: lightyellow;
    background-image: url("bg.jpg");
    background-size: cover;
    opacity: 0.8;
}
```

9.2 Text and Fonts

```
p {
    font-family: Arial, sans-serif;
```

```
    font-size: 16px;
    font-weight: bold;      /* normal | bold | 100-900 */
    font-style: italic;
    text-align: center;     /* left | right | center | justify */
    text-decoration: underline;
    text-transform: uppercase;
    letter-spacing: 2px;
    line-height: 1.6;
}
```

9.3 Box Model

Every element is a box. From outside in: margin, border, padding, content.

```
div {
    width: 300px;
    height: 150px;
    padding: 20px;              /* space inside border     */
    border: 2px solid black;
    border-radius: 8px;         /* rounded corners         */
    margin: 10px auto;          /* space outside border; auto centers */
    box-shadow: 2px 2px 5px gray;
}
```

9.4 Display and Visibility

```
div  { display: block; }       /* takes full width           */
span { display: inline; }      /* sits inside a line of text */
.box { display: inline-block;} /* inline but accepts width/height */
.hidden { display: none; }     /* removed from layout        */
.invisible { visibility: hidden; } /* hidden but still takes space */
```

9.5 Positioning

```
.normal   { position: static; }      /* default; follows page flow  */
.relative { position: relative;      /* offset from its normal spot */
            top: 10px; left: 20px; }
.absolute { position: absolute;      /* removed from flow, placed   */
            top: 50px; left: 100px;} /* relative to nearest parent  */
.fixed    { position: fixed;         /* stays in place on scroll    */
            bottom: 0; right: 0; }
.sticky   { position: sticky;        /* scrolls then sticks         */
            top: 0; }
```

9.6 Flexbox (Simple Layout)

```
.container {
    display: flex;
    justify-content: center;    /* horizontal alignment */
    align-items: center;        /* vertical alignment   */
    gap: 16px;
}
```

9.7 CSS Variables

```
:root {
    --main-color: #3498db;
    --font-size: 16px;
}

p {
    color: var(--main-color);
    font-size: var(--font-size);
}
```

Part III

jQuery

10 What is jQuery?

jQuery is a JavaScript library that makes it much easier to interact with HTML and CSS from code. Instead of several lines of vanilla JavaScript, most tasks take a single line in jQuery.

11 Basic Syntax

```
$(selector).action();
```

- `$` – shorthand for `jQuery`
- *selector* – finds element(s), same syntax as CSS
- *action* – what to do: change text, add class, hide, etc.

Always wrap code inside the ready callback:

```
$(document).ready(function() {
    // runs after page is fully loaded
});

// shorthand
$(function() { ... });
```

12 Connecting jQuery to HTML and CSS

```
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="style.css">
    <!-- Load jQuery BEFORE your own script -->
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
    <p id="para" class="text">Hello World</p>
```

```
    <button id="btn">Click Me</button>

    <script>
    $(function() {
        $("#btn").click(function() {
            $("#para").text("Changed!");
            $("#para").css("color", "blue");
        });
    });
    </script>
</body>
</html>
```

13 jQuery Selectors

jQuery selectors use the exact same syntax as CSS selectors:

```
$("p")                    // all <p> elements
$("#para")                // element with id="para"
$(".text")                // all elements with class="text"
$("p.text")               // <p> elements that also have class="text"
$("div p")                // all <p> inside a <div>
$("input[type='text']")   // text inputs (attribute selector)
$("li:first")             // first <li>
$("tr:even")              // even-indexed rows (great for striping)
$("p:contains('Hello')")  // <p> containing the word Hello
```

14 Changing Content and Styles

14.1 Text and HTML

```
$("#para").text("New plain text");        // set plain text
$("#para").text();                        // read plain text
$("#para").html("<b>Bold content</b>");   // set with HTML tags
$("#para").html();                        // read inner HTML
```

14.2 CSS Styles

```
$("#para").css("color", "red");            // set one property
$("#para").css("color");                   // read a property

$("#para").css({                           // set multiple at once
    "color": "white",
    "background-color": "navy",
    "font-size": "20px"
});
```

14.3 Adding and Removing CSS Classes

```
$("#para").addClass("highlight");
$("#para").removeClass("highlight");
$("#para").toggleClass("highlight");   // on if off, off if on

if ($("#para").hasClass("highlight")) {
    console.log("Class is present");
}
```

15 Adding and Removing Elements

```
<div id="container"></div>
<button id="addBtn">Add</button>
<button id="removeBtn">Remove Last</button>
```

```
$("#addBtn").click(function() {
    $("#container").append("<p>New paragraph</p>");   // add at end
    $("#container").prepend("<p>At the start</p>");   // add at top
});

$("#removeBtn").click(function() {
    $("#container p:last").remove();
});
```

16 Event Handling

```
$("#btn").click(function() { alert("Clicked!"); });

$("#box").hover(
    function() { $(this).css("background", "lightblue"); },  // mouse in
    function() { $(this).css("background", "white"); }       // mouse out
);

$("#input").keyup(function() {
    console.log($(this).val());   // print value on every key release
});

// .on() works for any event, including elements added later
$("#container").on("click", "p", function() {
    $(this).css("color", "purple");
});
```

17 DOM Traversal

```
$("#child").parent();          // direct parent
$("#list").children();         // direct children
$("#list").find("li");         // all matching descendants
$("#item").siblings();         // all siblings
$("#item").next();             // next sibling
$("#item").prev();             // previous sibling
```

18 Attribute and Property Manipulation

```
// Attributes
$("#logo").attr("src");              // read
$("#logo").attr("src", "new.png");   // set
$("#logo").removeAttr("alt");        // remove

// Properties (boolean)
$("#chk").prop("checked");           // true or false
$("#chk").prop("checked", true);     // check the checkbox
$("#btn").prop("disabled", true);    // disable a button
```

19 Form Handling

```
// Read any input value
var name = $("#username").val();

// Validate on submit
$("#myForm").submit(function(event) {
    event.preventDefault();
    var name = $("#username").val().trim();
    if (name === "") {
        alert("Name cannot be empty!");
    } else {
        alert("Hello, " + name);
    }
});

// Checkbox
if ($("#agree").is(":checked")) {
    console.log("Agreed");
}

// Radio
var plan = $("input[name='plan']:checked").val();

// Dropdown
$("#country").val();                 // get selected
$("#country").val("pk");             // set selected
$("#country").change(function() {
    console.log($(this).val());
});
```

20 Table Handling

```
// Add a row
$("#addRow").click(function() {
    $("#myTable").append("<tr><td>Sara</td><td>22</td></tr>");
});

// Delete a row (delete button inside each row)
$("#myTable").on("click", ".deleteBtn", function() {
```

```
    $(this).closest("tr").remove();
});

// Highlight clicked row
$("#myTable tr").click(function() {
    $(this).toggleClass("highlight");
});

// Loop through all rows
$("#myTable tr").each(function(index) {
    console.log("Row " + index + ": " + $(this).text());
});

// Row count
var total = $("#myTable tbody tr").length;
console.log("Total rows: " + total);
```

21 Chaining

Multiple jQuery methods can be strung together on one line:

```
$("#para")
    .text("Hello!")
    .css("color", "white")
    .addClass("highlight")
    .css("font-size", "22px");
```

22 jQuery vs. Vanilla JavaScript

Vanilla JavaScript

```
document.getElementById("para").innerText = "Hi";
document.getElementById("para").style.color = "red";
document.getElementById("btn").addEventListener("click", function() {
    alert("Clicked!");
});
var items = document.querySelectorAll("li");
items.forEach(function(item) {
    item.style.color = "blue";
});
```

Same thing in jQuery

```
$("#para").text("Hi");
$("#para").css("color", "red");
$("#btn").click(function() { alert("Clicked!"); });
$("li").css("color", "blue");
```

23 Common Methods Cheat Sheet

Method	Purpose
`.text()`	Get/set plain text content
`.html()`	Get/set HTML content
`.css()`	Get/set CSS style
`.addClass()`	Add a CSS class
`.removeClass()`	Remove a CSS class
`.toggleClass()`	Toggle a CSS class on/off
`.hasClass()`	Check if a class exists
`.append()`	Insert content at the end
`.prepend()`	Insert content at the start
`.remove()`	Remove an element from the page
`.val()`	Get/set form field value
`.attr()`	Get/set HTML attribute
`.prop()`	Get/set boolean property
`.click()`	Attach a click event handler
`.on()`	Attach any event (incl. future elements)
`.each()`	Loop through matched elements
`.find()`	Search descendants
`.closest()`	Find nearest matching ancestor

Table 1: Frequently used jQuery methods

24 Summary

- **HTML** provides the structure and content of a page.
- **CSS** controls how that content looks – colors, fonts, layout.
- **jQuery** controls how the page *behaves* – what happens when the user clicks, types, or submits a form – by reading and changing HTML and CSS through short, chainable method calls.